# Herramientas de Big Data

Taller 1: Big Data Tools - Workshop 1

**Estudiantes:** Fermin Escalona (300181), Julian Romero (325312)
**Carrera:** Ing. Informática, Ing. Informática
Universidad de la Sabana
Maestría en Analítica Aplicada
**Profesor:** Hugo Franco, Ph.D.

6 de abril de 2026

---

## 1. Advanced Filesystem & Format Diagnostics

**1.1. Descripción del problema:** La capacidad de identificar la ubicación de los datos y evaluar el consumo de recursos es fundamental en la ingeniería de datos. El objetivo es desarrollar herramientas de diagnóstico para el sistema de archivos, recuperando la ruta absoluta de un archivo `.csv` y verificando si el entorno de Python posee permisos de lectura y escritura sobre el mismo. Adicionalmente, se requiere determinar el espacio total, usado y libre de la partición del disco, aplicando una conversión dinámica de bytes a unidades legibles (KB, MB o GB).

**1.2. Método de solución:** El problema se aborda mediante un conjunto de funciones de diagnóstico definidas en `utils.py`. Para los archivos, se utiliza la librería estándar del sistema operativo para consultar la ruta absoluta y verificar las banderas de acceso en lectura y escritura. Para el almacenamiento, se extraen las métricas de uso del disco y se procesan a través de un algoritmo de conversión iterativo que escala la unidad de medida según el tamaño del archivo. Como se observa en el Algoritmo 1, este enfoque se divide en tres procedimientos auxiliares.

Listing 1: Diagnostico de Archivos y Sistema

```
funcion get_file_diagnostics: tupla(cadena, booleano, booleano)
  Parametro: filename (cadena)

  ruta_absoluta = ObtenerRutaAbsoluta(filename)
  permiso_lectura = VerificarAcceso(ruta_absoluta, MODO_LECTURA)
  permiso_escritura = VerificarAcceso(ruta_absoluta, MODO_ESCRITURA)

  retornar (ruta_absoluta, permiso_lectura, permiso_escritura)

funcion get_disk_diagnostics: tupla(cadena, cadena, cadena)
  Parametro: path (cadena)

  uso_disco = ObtenerUsoDeDisco(path)
  total = format_bytes(uso_disco.total)
  usado = format_bytes(uso_disco.usado)
  libre = format_bytes(uso_disco.libre)

  retornar (total, usado, libre)
```

```
funcion format_bytes: cadena
  Parametro: size_bytes (num rico)

  Si size_bytes es igual a 0:
    retornar "0 B"

  unidades = Arreglo("B", "KB", "MB", "GB", "TB")
  indice = 0

  Mientras size_bytes >= 1024 y indice < longitud(unidades) - 1:
    size_bytes = size_bytes / 1024
    indice = indice + 1

  retornar size_bytes formateado con 2 decimales concatenado con
   unidades[indice]
```

**1.3. Resultados:** Al ejecutar el script `main.py`, el sistema identificó exitosamente la ruta absoluta del archivo en el directorio local del usuario y verificó que se cuenta con permisos tanto de lectura como de escritura. Con respecto al almacenamiento, como se observa en la Figura 1, el programa detectó un disco con una capacidad total de 1.81 TB, reportando 148.56 GB de espacio usado y 1.67 TB de espacio libre. Esto demuestra el correcto funcionamiento de la conversión dinámica de unidades.

```
--- 1. DIAGNÓSTICO AVANZADO DE SISTEMA DE ARCHIVOS ---
Ruta absoluta: /Users/fermin/Documents/GitHub/BDT_Biomechanical_Workshop/result_retrieve_left-and-right_x_50_2016_workshop.csv
Permiso de lectura: SÍ
Permiso de escritura: SÍ

Espacio en disco:
  Total: 1.81 TB
  Usado: 148.56 GB
  Libre: 1.67 TB
```

Figura 1: Salida por consola del diagnóstico avanzado de sistema de archivos. Fuente: elaboración propia.

**1.4. Discusión:** Los resultados obtenidos permiten asegurar la disponibilidad técnica de los insumos. Garantizar permisos y capacidad de disco antes de iniciar un flujo de procesamiento es una medida de integridad que mitiga errores críticos de sistema durante operaciones masivas de entrada/salida sobre grandes volúmenes de datos.

# 2. Data Exploration & Cleansing

**2.1. Descripción del problema:** Los conjuntos de datos del mundo real requieren inspecciones exhaustivas más allá del conteo básico de nulos. Esta sección exige la validación de los tipos de datos para identificar anomalías y el cálculo del porcentaje de valores faltantes por columna. Finalmente, se debe asegurar la integridad del dataset mediante la remoción de registros incompletos específicamente en las columnas `value_x`, `value_y` y `value_z`.

**2.2. Método de solución:** El método carga los datos en un objeto tabular (DataFrame) y extrae sus metadatos de tipado estructural. Para la cuantificación de faltantes, se implementa una función matemática que determina el peso porcentual de los valores nulos sobre el total de registros de cada columna, tal como se documenta en el Algoritmo

2. La limpieza posterior excluye mediante filtrado las filas que presenten anomalías en las columnas críticas.

Listing 2: Limpieza y Exploracion de Datos

```
funcion get_missing_percentages: Serie (numerica)
  Parametro: df (DataFrame)

  total_filas = Longitud(df)
  nulos_por_columna = SumarValoresNulosPorColumna(df)

  porcentaje_nulos = (nulos_por_columna / total_filas) * 100

  retornar porcentaje_nulos
```

**2.3. Resultados:** Inicialmente, la exploración de datos expuesta en la Figura 2 reveló un dataset compuesto por 13,621 filas y 20 columnas. El análisis de integridad identificó que las variables críticas (`value_x`, `value_y`, `value_z`) presentaban aproximadamente un 8.78 % de valores nulos. Tras aplicar el método de limpieza, el nuevo dataset se redujo a 12,425 registros válidos, tal como corrobora la Figura 3, evidenciando una reducción absoluta al 0.0 % en la tasa de valores faltantes para dichas columnas.

```
--- 2. EXPLORACIÓN Y LIMPIEZA DE DATOS ---

Dimensiones del dataset: (13621, 20)
Tipos de datos por columna:
subject_id        str
date              str
otp               str
trial           int64
group             str
marker            str
side              str
joint             str
variable          str
units             str
protocol          str
value_x       float64
value_y       float64
value_z       float64
sd_x          float64
sd_y          float64
sd_z          float64
md_x          float64
md_y          float64
md_z          float64
dtype: object

Porcentaje de valores nulos por columna:
subject_id    0.000000
date          0.000000
otp           0.000000
trial         0.000000
group         0.000000
marker        0.000000
side          0.000000
joint         0.000000
variable      0.000000
units         0.007342
protocol      0.007342
value_x       8.780559
value_y       8.780559
value_z       8.780559
sd_x          0.007342
sd_y          0.007342
sd_z          0.007342
md_x          0.007342
md_y          0.007342
md_z          0.007342
dtype: float64
```

Figura 2: Exploración de datos, dimensiones iniciales y porcentajes de valores nulos. Fuente: elaboración propia.

```
Limpieza completada:
Filas originales: 13621
Filas tras limpieza: 12425

Porcentaje de valores nulos en el dataset limpio:
subject_id    0.0
date          0.0
otp           0.0
trial         0.0
group         0.0
marker        0.0
side          0.0
joint         0.0
variable      0.0
units         0.0
protocol      0.0
value_x       0.0
value_y       0.0
value_z       0.0
sd_x          0.0
sd_y          0.0
sd_z          0.0
md_x          0.0
md_y          0.0
md_z          0.0
dtype: float64
```

Figura 3: Dimensiones del dataset tras la limpieza y confirmación de remoción de valores nulos. Fuente: elaboración propia.

**2.4. Discusión:** La exclusión rigurosa de las observaciones inconsistentes es esencial para que las fases de cálculo matemático y agregación estadística no se sesguen debido a la propagación de valores indefinidos.

# 3. Performance-oriented Data Manipulation

**3.1. Descripción del problema:** Es necesario evaluar y contrastar el impacto computacional de distintas metodologías algorítmicas. Para ello, se debe generar una nueva columna `composite_score` utilizando la fórmula matemática $(value_x * 0{,}5) + (value_y * 0{,}3) + value_z$ a través de tres técnicas: procesamiento iterativo (fila a fila), funcional (aplicación lambda) y vectorizado, cronometrando sus ejecuciones.

**3.2. Método de solución:** En el script principal, se configuran cronómetros para aislar el coste computacional de cada técnica. Se emplea una lista de anexión secuencial para la iteración manual, una función anónima mapeada en un eje para la aplicación funcional, y aritmética de matrices para la vectorización, como se expone en el Algoritmo 3.

Listing 3: Medicion de Rendimiento y Manipulacion

```
procedimiento Medir_Rendimiento_y_Manipulacion
  Parametro: df_cleaned (DataFrame)

  // Metodo 1: Iteracion fila por fila (Anti-patron)
  inicio_loop = ObtenerTiempoActual()
  scores_loop = ListaVacia()
  Para cada (indice, fila) en df_cleaned:
    score = (fila['value_x'] * 0.5) + (fila['value_y'] * 0.3) + fila['
   value_z']
```

```
    Agregar score a scores_loop
  df_cleaned['score_loop'] = scores_loop
  tiempo_loop = ObtenerTiempoActual() - inicio_loop

  // Metodo 2: Aplicacion Funcional (df.apply)
  inicio_apply = ObtenerTiempoActual()
  df_cleaned['score_apply'] = AplicarFuncionFilaAFila(df_cleaned,
      fila -> (fila['value_x'] * 0.5) + (fila['value_y'] * 0.3) + fila['
   value_z'])
  tiempo_apply = ObtenerTiempoActual() - inicio_apply

  // Metodo 3: Vectorizacion (El est ndar)
  inicio_vector = ObtenerTiempoActual()
  df_cleaned['composite_score'] = (df_cleaned['value_x'] * 0.5) +
                                  (df_cleaned['value_y'] * 0.3) +
                                  df_cleaned['value_z']
  tiempo_vector = ObtenerTiempoActual() - inicio_vector

  Imprimir(tiempo_loop, tiempo_apply, tiempo_vector)
```

**3.3. Resultados:** La evaluación de rendimiento documentada en la Figura 4 demuestra diferencias computacionales abismales. El método iterativo clásico (`iterrows`) tardó 0.115973 s en procesar las filas. La aplicación funcional (`df.apply`) redujo el tiempo a 0.032851 s. Finalmente, la vectorización directa sobre las series demostró ser muy superior, completando la tarea en apenas 0.000194 s.

```
--- 3. PERFORMANCE-ORIENTED DATA MANIPULATION ---
Tiempo iterrows:    0.115973 s
Tiempo df.apply:    0.032851 s
Tiempo Vectorización: 0.000194 s
```

Figura 4: Comparación de tiempos de ejecución para los métodos de iteración, función y vectorización. Fuente: elaboración propia.

**3.4. Discusión:** El análisis demuestra empíricamente que las rutinas iterativas de Python generan latencias excesivas por la sobrecarga del entorno, mientras que las operaciones vectorizadas aprovechan rutinas optimizadas en bajo nivel, agilizando enormemente la manipulación de grandes matrices numéricas. Reafirmando la potencia de librerías como pandas sobre el procesamiento de grandes volumenes de datos.

# 4. Database Integration & Retrieval

**4.1. Descripción del problema:** Tras la finalización del proceso de limpieza, la información requiere persistencia estructurada para su posterior recuperación. Los datos deben inyectarse de forma eficiente en una base de datos PostgreSQL alojada en un contenedor. Finalmente, se debe efectuar una consulta SQL parametrizada orientada a extraer un subconjunto de columnas para visualización, filtrando por articulaciones (Cadera, Rodilla, Tobillo), sujeto, fecha y protocolo.

**4.2. Método de solución:** La persistencia elude las inserciones manuales, optando por una carga masiva mediante el protocolo `COPY` (volcado directo desde memoria/ar-

chivo). La extracción de datos delega el filtrado al motor SQL utilizando predicados de igualdad y condicionales lógicos `IN`, lo que se modela a continuación.

Listing 4: Integracion y Consulta a Base de Datos

```
funcion get_db_connection: ObjetoConexion
  Intentar:
    conexion = ConectarBD(Host, Puerto, BaseDatos, Usuario, Clave)
    retornar conexion
  Excepcion e:
    Imprimir("Error al conectar")
    retornar Nulo

procedimiento load_data_to_postgres
  Parametros: df (DataFrame), table_name (cadena)

  conexion = get_db_connection()
  cursor = conexion.ObtenerCursor()

  cursor.Ejecutar("CREATE TABLE IF NOT EXISTS table_name (...)")
  cursor.Ejecutar("TRUNCATE TABLE table_name")

  Guardar df como archivo temporal "temp_data.csv" sin encabezados
  Abrir "temp_data.csv" como archivo_csv:
    sql = "COPY table_name FROM STDIN WITH (FORMAT CSV)"
    cursor.EjecutarCargaMasivaExperta(sql, archivo_csv)

  conexion.ConfirmarCambios()
  BorrarArchivo("temp_data.csv")

procedimiento Recuperacion_Consultas
  query = """
    SELECT side, joint, value_x, value_y, value_z
    FROM kinematics_data
    WHERE joint IN ('Hip', 'Knee', 'Ankle')
      AND subject_id = 'SUJETO_ESPECIFICO'
      AND trial = 10
      AND date = 'FECHA'
      AND protocol = 'M'
  """
  conexion = get_db_connection()
  df_plot = EjecutarConsultaSQL_A_DataFrame(query, conexion)
  retornar df_plot
```

**4.3. Resultados:** La ingesta masiva se completó sin errores y la consulta SQL recuperó la información esperada. Como ilustra la Figura 5, los primeros 5 registros devueltos por el motor relacional demuestran que los datos están correctamente filtrados, mostrando los valores de los ejes correspondientes exclusivamente a la Rodilla (`Knee`) para el lado izquierdo (`L`), lo que confirma la correcta formulación de las condicionales en el código.

```
Datos recuperados de la DB para graficar:
  side joint     value_x     value_y    value_z
0    L  Knee -165.694494 -123.182720  36.022345
1    L  Knee  -30.280983 -115.487058  46.915905
2    L  Knee  150.967856  -63.342459  45.734985
3    L  Knee  341.777878   21.205594  33.002680
4    L  Knee  504.822778  112.502710  14.049575
```

Figura 5: Visualización por consola del DataFrame resultante tras la consulta a PostgreSQL. Fuente: elaboración propia.

**4.4. Discusión:** El uso de cargas masivas desde archivos temporales delimitados es la mejor práctica de ingesta de Big Data en sistemas relacionales, transformando esos datos en información estructurada, segura y útil. A su vez, delegar el filtrado al motor de base de datos a través de una consulta SQL dirigida evita desbordamientos de memoria en el entorno de aplicación.

# 5. Data Visualization

**5.1. Descripción del problema:** Se requiere generar una representación visual de los datos biomecánicos basada en los ángulos articulares extraídos de la base de datos. La visualización debe estructurar un lienzo gráfico vertical segregando Cadera (Hip), Rodilla (Knee) y Tobillo (Ankle). Adicionalmente, se deben superponer y categorizar por color los ejes anatómicos izquierdo (Rojo) y derecho (Azul) en la misma gráfica.

**5.2. Método de solución:** Se inicializa un plano multiregión y se itera sobre la lista de articulaciones objetivo. Por cada articulación, se particiona el dataframe temporalmente por lado, y se trazan las señales para los tres ejes cartesianos (X, Y, Z) diferenciando la topología de la línea.

Listing 5: Visualizacion Biomecanica de Datos

```
procedimiento Generar_Graficos_Biomecanicos
  Parametro: df_plot (DataFrame)

  figura, ejes = CrearSubgraficos(filas=3, columnas=1, tama o=(10, 15))
  articulaciones = Arreglo('Hip', 'Knee', 'Ankle')

  Para i desde 0 hasta longitud(articulaciones) - 1:
    articulacion = articulaciones[i]

    // Filtros por lado
    subset_izquierdo = Filtrar(df_plot, joint == articulacion Y side ==
   'L')
    subset_derecho = Filtrar(df_plot, joint == articulacion Y side == 'R
   ')

    // Renderizacion Lado Izquierdo (Rojo)
    ejes[i].TrazarLinea(subset_izquierdo['value_x'], color='red', estilo
   ='-')
    ejes[i].TrazarLinea(subset_izquierdo['value_y'], color='red', estilo
   ='--')
```

```
    ejes[i].TrazarLinea(subset_izquierdo['value_z'], color='red', estilo
    =':')

    // Renderizacion Lado Derecho (Azul)
    ejes[i].TrazarLinea(subset_derecho['value_x'], color='blue', estilo=
    '-')
    ejes[i].TrazarLinea(subset_derecho['value_y'], color='blue', estilo=
    '--')
    ejes[i].TrazarLinea(subset_derecho['value_z'], color='blue', estilo=
    ':')

    // Metadatos del grafico
    ejes[i].DefinirTitulo('Joint Angles: ' + articulacion)
    ejes[i].DefinirEtiquetaY('Angle (degrees)')
    ejes[i].MostrarLeyenda(ubicacion='upper right')
    ejes[i].HabilitarCuadricula()

  AjustarDise oGlobal()
  MostrarGraficos()
```

**5.3. Resultados:** El script generó exitosamente una figura subdividida en un esquema vertical de 3x1 (Figura 6). En cada plano cartesiano se aprecian las series temporales que describen el comportamiento de los ángulos para las articulaciones analizadas. La leyenda confirma la superposición del miembro izquierdo en color rojo frente al miembro derecho en color azul, discriminando cada plano de movimiento (X, Y, Z) con un estilo de línea específico (continua, segmentada, punteada).

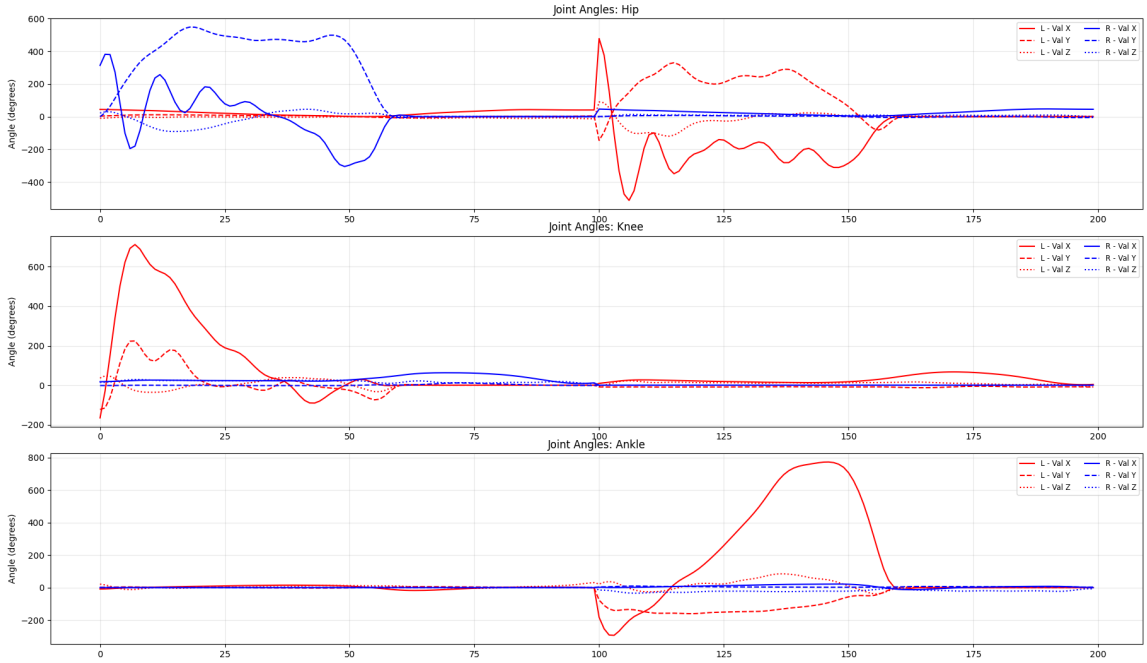


Figura 6: Visualización de los ángulos articulares (Cadera, Rodilla, Tobillo) diferenciados por extremidad y eje anatómico. Fuente: elaboración propia.

**5.4. Discusión:** El empaquetamiento de los planos en una sola visualización bajo escalas uniformes permite establecer diferencias bilaterales precisas. Esta abstracción técnica entrega valor clínico fundamental para detectar asimetrías articulares durante ciclos de movimientos analizado en el experimento.

# Referencias


- Franco, H. *Big Data Tools - Workshop 1*. Documento Técnico del Taller. Maestría en Analítica Aplicada, Universidad de la Sabana.
- Franco, H. *Especificación Taller HBD*. Plantilla Estructural para Entregas de Informes de Herramientas de Big Data. Universidad de la Sabana.